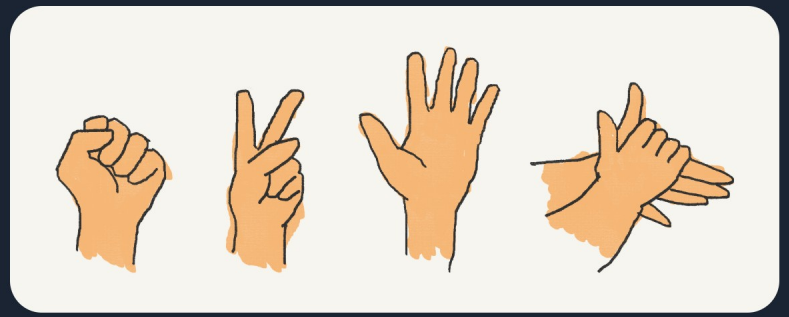
エンジニア研修 ・ オブジェクト指向 開発演習

じゃんけん演習

まとめて書いてから、
責務を見つけて分ける



責務の分割　カプセル化（不変条件）　多態性

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

01

ルール

今回のじゃんけんを知る

ルール

出せる手は4種類

いつものグー・チョキ・パーに、**ジャッカル**を加えた4種類の手で遊びます。手は、それぞれの英語名の頭文字で入力します。



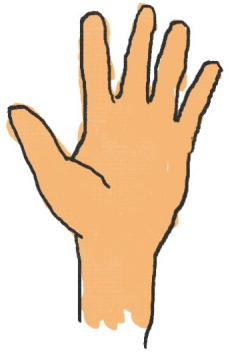
グー

`[r]ock`



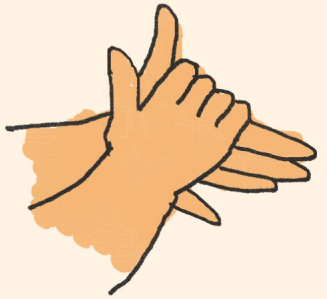
チョキ

`[s]cissors`



パー

`[p]aper`



ジャッカル

`[j]ackal`

ルール

ジャッカルは、チョキにもパーにも勝つ

2000年代に放送されたアニメに登場した、じゃんけんの手です。「チョキの5倍の威力」があり、チョキにもパーにも勝ちます。ただし、**グーには負けます**。



ジャッカルの勝ち



ジャッカルの勝ち

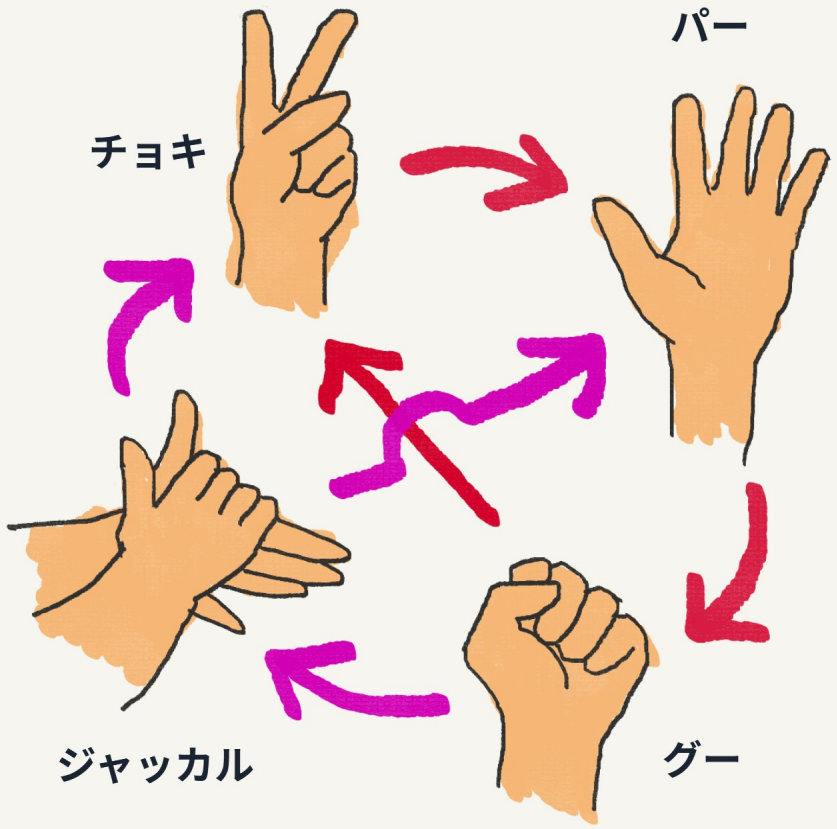


ジャッカルの負け

絵の中で薄く描いた手が、負けた手です。

ルール

手どうしの勝ち負け



矢印は「矢印の元の手が、矢印の先の手に勝つ」ことを表します。赤の矢印はグー・チョキ・パーどうし、赤紫の矢印はジャッカルが関わる勝ち負けです。同じ手どうしでは、勝ち負けはつきません。

手	勝てる手	負ける手
グー	チョキ、ジャッカル	パー
チョキ	パー	グー、ジャッカル
パー	グー	チョキ、ジャッカル
ジャッカル	チョキ、パー	グー

グー・チョキ・パーの3つは、いつもどおり「三すくみ」の関係です。

ルール

参加者は、プレイヤーとNPC

プレイヤー（人）

画面に表示される案内に従って、キーボードから手を入力します。

```
[r]ock, [p]aper, [s]cissors, [j]ackal > r
```

NPC（コンピューター）

4種類の手から、毎回ランダムに1つを選んで出します。

参加者は合計2名以上。何人でも参加できます。

人数は起動するときに指定します。指定しなければ、プレイヤー1名とNPC1名で対戦します。

```
--player 1 --npc 1
```

ルール

ゲームの流れ

1

全員が一斉に手を出す

プレイヤーは入力し、NPCはランダムに選びます。

2

負けた人が抜ける

ただし、全員が負けたときは、誰も抜けません。

3

勝者が決まるまで繰り返す

残った人で、もう一度1から行います。

次のページから、実際の対戦の例を見ていきます。

ルール ・ 対戦の例

例1：2人で対戦する

Aさん



グー

残る

Bさん



チョキ

抜ける

結果：Bさんが抜けて、Aさんの勝ち

ルール ・ 対戦の例

例2：3人で対戦する

Aさん



グー

残る

Bさん



チョキ

抜ける

Cさん



チョキ

抜ける

結果：BさんとCさんが抜けて、Aさんの勝ち

ルール ・ 対戦の例

例3：3人がバラバラの手を出す

Aさん



グー

残る

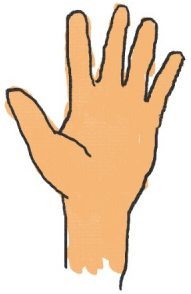
Bさん



チョキ

残る

Cさん



パー

残る

結果：全員が誰かに負けているので、誰も抜けない。3人でもう一度

ルール ・ 対戦の例

例4：4人で対戦する

Aさん



グー

残る

Bさん



グー

残る

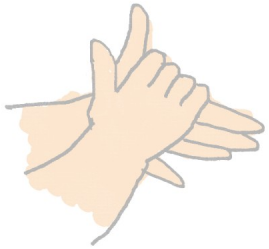
Cさん



チョキ

抜ける

Dさん



ジャッカル

抜ける

結果：CさんとDさんが抜ける。AさんとBさんの2人でもう一度

ルール ・ 対戦の例

例5：3人が同じ手を出す

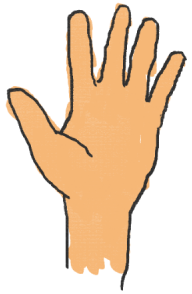
Aさん



パー

残る

Bさん



パー

残る

Cさん



パー

残る

結果：誰も負けていないので、誰も抜けない。3人でもう一度

02

進め方

まとめて書いてから、分ける

進め方

分け方を自分で決める

この演習では、作るクラスをあらかじめ決めません。まず動くものを作り、その流れを見ながら分け方を考えます。

1 はじめに、全体を動かす

すべての処理を main に書いて、ゲームを最後まで動かす

ゲームの流れ全体を、1か所で見渡せるようにする

2 流れを見ながら、分ける

どこで分けるかは、動くコードを見ながら自分たちで決める

分け方の正解は1つではない。なぜそう分けたかを説明できるようにする

先に全体を動かすと、何と何がつながっているかが見える。

進め方

4つの段階で分ける

一度に分けようとせず、小さな段階を順に進みます。どの段階でも、ゲームの動きは変えません。

1

main に書き下す

クラスもメソッドも作らず、すべてを main に書いて動かす

2

メソッドに切り出す

ひとまとまりの処理に、名前をつけて切り出す

3

クラスに切り出す

いつも一緒に使うデータと処理を、1つにまとめる

4

役割で分ける

種類ごとに違うところを、それぞれに任せる

名前をつける → まとめる → 任せる、の順に、少しずつ形を変えていく。

進め方

切り出す手がかり

コードの中に次のようなところを見つけたら、切り出すきっかけになります。

見つけたもの	切り出し方
同じ処理が、2か所以上に書かれている	メソッドにして、1か所にまとめる
コメントで「〜する」と説明しているかたまり	メソッドにする（コメントが名前の候補）
いつも一緒に受け渡ししているデータ	クラスにまとめる
同じデータを読み書きするメソッドの集まり	データと一緒に、クラスに移す
種類によって、if で書き分けている処理	種類ごとのクラスに任せる（多態性）

進め方

動きを変えずに分ける

切り出すたびに、**同じシードと同じ入力**で動かし、表示が変わらないことを確かめます。切り出す前の表示を before.txt に保存しておきます。

```
./mvnw -q compile
printf 'r\nr\nr\nr\n' | java -cp target/classes janken.JankenApp --player 1 --npc 2 --seed 26 > before.txt

# （切り出したあと）
./mvnw -q compile
printf 'r\nr\nr\nr\n' | java -cp target/classes janken.JankenApp --player 1 --npc 2 --seed 26 > after.txt
diff before.txt after.txt
```

ポイント	理由
--seed 26 を渡す	NPC が出す手が、毎回同じになる
printf で入力を渡す	プレイヤーが出す手が、毎回同じになる
diff で何も表示されない	切り出しの前後で、動きが変わっていない

※ シード（seed）：乱数（ランダムな値）を作るときの「元になる数」です。同じシードを渡すと、毎回同じ順番で同じ乱数が出てきます。

03

課題

書いて、分けて、確かめる

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 課題1

ルールの中にある法則を見つけよう

3〜4人で以下を話し合いましょう。

1. 「参加者」が勝つ、とはどういう状態か

2. もう一度じゃんけんをする（あいこ）、とはどういう状態か

3. 次の回に残れる、とはどういう状態か。全員が誰かに負けたときはどうなるか

4. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
   （それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

ヒント：2人・3人・4人での対戦の例を思い出しましょう。4は、ルールの説明に出てくる名詞と動詞を拾い出すところから始めます。

検討例 ・ 課題1

勝ち・あいこ・残る・もの を、言葉で言い表す

1. 「参加者」が勝つ、とはどういう状態か

→ 最後の一人になること

2. もう一度じゃんけんをする（あいこ）、とはどういう状態か

→ 二人以上が残っていること。人数が減っても、一人に決まるまではあいこ

3. 次の回に残れる、とはどういう状態か。全員が誰かに負けたときはどうなるか

→ 場に自分より強い手が出ていないこと（負けないこと）。全員が負けたら、全員が残る

4. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
（それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

→ 手（どの手に負けるかを知る）、参加者（手を出す）、じゃんけん（勝者が決まるまで繰り返す）

検討例 ・ 課題1

見つけた法則が、分け方の手がかりになる

対象	見つけた法則	分けるときに考えること
参加者	勝つ ＝ 最後の一人になる	一人になるまで回を繰り返す
手	残れる ＝ 自分より強い手が場にない	どの手に負けるかを、手自身に持たせる
参加者	例外：全員が負けたら全員残る	残る人を決める処理を、1か所にまとめる
参加者	プレイヤーとNPCは、手の出し方だけが違う	出し方だけを、それぞれに任せる
もの	手・参加者・じゃんけん・起動の設定	名詞はクラスの、動詞はメソッドの候補

いきなりクラスを決めなくてよい。まず main に書き、ここで見つけた「もの」を思い出しながら分けていく。

Maven Wrapper を取得する

Maven Wrapper を直接ダウンロードし、使う Maven のバージョンを指定します。

```
WRAPPER_VERSION=3.3.4
MAVEN_VERSION=3.10.0

mkdir janken13 && cd janken13

curl -fsSLO "https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper-
distribution/${WRAPPER_VERSION}/maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip"
unzip -l maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
unzip maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
rm maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip

mkdir -p .mvn/wrapper
cat > .mvn/wrapper/maven-wrapper.properties <<EOF
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/${MAVEN_VERSION}/apache-
maven-${MAVEN_VERSION}-bin.zip
EOF

chmod +x mvnw
```

準備 ・ プロジェクトを用意する（2/3）

Maven Wrapper でひな形を作る

取得した mvnw で、Maven 公式のテンプレート（archetype）からプロジェクトを生成します。janken13 フォルダの中で実行してください。

```
./mvnw archetype:generate \
  -DarchetypeGroupId=org.apache.maven.archetypes \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DjavaCompilerVersion=25 \
  -DgroupId=com.example \
  -DartifactId=janken \
  -Dpackage=janken \
  -DinteractiveMode=false

mv janken/* ./ && rm -rf janken/
```

archetype:generate は、artifactId と同じ名前（janken）のフォルダを作って、その中にプロジェクトを生成します。最後の行で、生成された中身を janken13 の直下に移しています。

-DjavaCompilerVersion=25 で、使う Java のバージョンを 25 に指定しています（指定しないと 17 になります）。あらかじめ JDK 25 をインストールしておいてください。

準備 ・ プロジェクトを用意する（3/3）

.gitignore を作る

ビルド結果や IDE の設定ファイルを、Git の管理から外します。janken13 フォルダの中で実行してください。

```
cat > .gitignore <<'EOF'
target/
.idea/
*.iml
.vscode/
.DS_Store
mvnwDebug*
before.txt
after.txt
EOF
```

できあがる構成

```
janken13/
├── .gitignore
├── .mvn/wrapper/
│   └── maven-wrapper.properties
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src/
    ├── main/java/janken/
    └── test/java/janken/
```

.mvn フォルダは Git の管理から外さないでください。使う Maven のバージョンが書かれています。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 課題2

main に書き下す

JankenApp クラスの main メソッドの中に、ゲームのすべてを書いて動かします。まずは動けばよく、きれいに書く必要はありません。

1. 新しいクラスもメソッドも作らず、main 1つの中に書く
2. 手は int（0〜3 の番号）や String で表してよい
3. 起動引数 --player、--npc、--seed を読み取り、NPC の手は new Random(シード) で選ぶ
4. 画面の表示は、次のページの表示例に合わせる

完成したら、課題3以降で比べるために before.txt を作っておく（進め方「動きを変えずに分ける」のコマンド）

演習 ・ 課題2

画面の表示例

プレイヤー1名、NPC2名で対戦した例です。画面の表示はすべて英語にし、手は Rock・Paper・Scissors・Jackal と表示します。

```
$ java -cp target/classes janken.JankenApp --player 1 --npc 2 --seed 26

--- Round 1 (3 players left) ---
Player1, choose your hand [r]ock, [p]aper, [s]cissors, [j]ackal > r
  Player1: Rock
  Npc1: Scissors
  Npc2: Rock
Draw: remaining [Player1, Npc2]

--- Round 2 (2 players left) ---
Player1, choose your hand [r]ock, [p]aper, [s]cissors, [j]ackal > r
  Player1: Rock
  Npc2: Scissors

The winner is Player1!
```

人数が条件（0以上で、合計2名以上）を満たさないときは、使い方（Usage）を表示して終わります。

検討例 ・ 課題2

main の中身を、かたまりで見る

書き上げた main を上から読むと、次のかたまりが並んでいます。

かたまり	やっていること
起動引数を読む	人数とシードを読み取り、条件を満たさなければ使い方を表示する
参加者を並べる	Player1, Player2, …, Npc1, Npc2, … の名前を並べる
全員が手を出す	名前が Player で始まれば入力、そうでなければランダムに選ぶ
残る人を決める	自分より強い手が出ていなければ残る。全員が負けたら全員が残る
繰り返す・表示する	一人になるまで回を繰り返し、最後に勝者を表示する

手どうしの勝ち負けは、beats[a][b]（手 a は手 b に勝つ）のような表にすると書きやすい

解答例 ・ 課題2（1/3）

main に書き下した例

JankenApp.java

```
public class JankenApp {
    public static void main(String[] args) {
        // 起動引数を読み取る
        int players = 1;
        int npcs = 1;
        long seed = System.nanoTime();
        for (int i = 0; i + 1 < args.length; i += 2) {
            if (args[i].equals("--player")) {
                players = Integer.parseInt(args[i + 1]);
            } else if (args[i].equals("--npc")) {
                npcs = Integer.parseInt(args[i + 1]);
            } else if (args[i].equals("--seed")) {
                seed = Long.parseLong(args[i + 1]);
            }
        }
        if (players < 0 || npcs < 0 || players + npcs < 2) {
            System.out.println("Usage: JankenApp [--player N]"
                    + " [--npc N] [--seed N]");
            System.out.println("  N >= 0, 2 or more players in total");
            return;
        }
        Random random = new Random(seed);
        Scanner scanner = new Scanner(System.in);

        // 手：0 = Rock, 1 = Paper, 2 = Scissors, 3 = Jackal
```

JankenApp.java（続き）

```
        String[] names = {"Rock", "Paper", "Scissors", "Jackal"};
        String[] keys = {"r", "p", "s", "j"};
        // beats[a][b] が true なら、手 a は手 b に勝つ
        boolean[][] beats = {
                {false, false, true, true},
                {true, false, false, false},
                {false, true, false, false},
                {false, true, true, false},
        };

        // 参加者を並べる
        List<String> members = new ArrayList<>();
        for (int i = 1; i <= players; i++) {
            members.add("Player" + i);
        }
        for (int i = 1; i <= npcs; i++) {
            members.add("Npc" + i);
        }

        int round = 1;
        while (members.size() > 1) {
            System.out.println();
            System.out.println("--- Round " + round + " ("
                    + members.size() + " players left) ---");
```

解答例 ・ 課題2（2/3）

main に書き下した例

JankenApp.java（続き）

```
            // 全員が手を出す
            List<Integer> hands = new ArrayList<>();
            for (String name : members) {
                int hand = -1;
                if (name.startsWith("Player")) {
                    while (hand < 0) {
                        System.out.print(name + ", choose your hand"
                                + " [r]ock, [p]aper, [s]cissors,"
                                + " [j]ackal > ");
                        String input = scanner.nextLine().trim();
                        for (int h = 0; h < keys.length; h++) {
                            if (keys[h].equals(input)) {
                                hand = h;
                            }
                        }
                    }
                } else {
                    hand = random.nextInt(names.length);
                }
                hands.add(hand);
            }
            for (int i = 0; i < members.size(); i++) {
                System.out.println("  " + members.get(i) + ": "
                        + names[hands.get(i)]);
            }
```

JankenApp.java（続き）

```
            // 自分より強い手が出ていなければ残る
            List<String> survivors = new ArrayList<>();
            for (int i = 0; i < members.size(); i++) {
                boolean lost = false;
                for (int other : hands) {
                    if (beats[other][hands.get(i)]) {
                        lost = true;
                    }
                }
                if (!lost) {
                    survivors.add(members.get(i));
                }
            }
            // 全員が負けたら、全員が残る
            if (survivors.isEmpty()) {
                survivors = new ArrayList<>(members);
            }
            members = survivors;
            if (members.size() > 1) {
                System.out.println("Draw: remaining " + members);
            }
            round++;
        }

        System.out.println();
```

解答例 ・ 課題2（3/3）

main に書き下した例

JankenApp.java（続き）

```
        System.out.println("The winner is " + members.get(0) + "!");
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 課題3

メソッドに切り出す

課題2の main から、ひとまとまりの処理を static メソッドとして切り出します。

1. main を読み、ひとまとまりの処理に「〜する」と名前をつける
2. 同じ処理が2か所にあれば、1つのメソッドにまとめる
3. 1つ切り出すごとにコンパイルして動かし、diff before.txt after.txt で表示が変わらないことを確かめる

条件：main を読むだけで、ゲームの流れ（回の始め → 全員が手を出す → 残る人を決める → 勝者の表示）がわかるようにする

検討例 ・ 課題3

メソッドに切り出す

役割	作るもの
起動引数から値を取り出す	option(args, name, defaultValue): String
使い方を表示する	printUsage(): void
参加者の名前を並べる	createMembers(players, npcs): List<String>
回の見出しを表示する	printRoundHeader(round, members): void
全員に手を出させる	chooseHands(members, scanner, random)
プレイヤーに手を入力してもらう	askHand(name, scanner): int
出した手を表示する	printHands(members, hands): void
手が場の誰かに負けているかを検査	isBeaten(hand, hands): boolean
残る人を決める	survivors(members, hands): List<String>

検討例 ・ 課題3 クラス図

同じデータを受け取るメソッドが集まっている

課題3のメソッドを、受け取るデータで分けてみます。いつも同じデータを受け取るメソッドは、そのデータと一緒にクラスにまとめる候補です。

受け取るデータ	メソッド	次の課題では
String[] args	option()	起動の設定（GameOptions）にまとめる
int hand と、場の手	isBeaten()	手（Hand）に持たせる
名前と手のリストを2本	printHands(), survivors()	「だれが・どの手」を組にする
名前（Player で始まるか）	chooseHands(), askHand()	課題5で役割に分ける

「どのデータを受け取るか」が、クラスの境目を教えてくれる。

解答例 ・ 課題3（1/3）

メソッドに切り出した例

JankenApp.java

```
public class JankenApp {
    private static final String[] NAMES =
            {"Rock", "Paper", "Scissors", "Jackal"};
    private static final String[] KEYS = {"r", "p", "s", "j"};
    private static final boolean[][] BEATS = {
            {false, false, true, true},
            {true, false, false, false},
            {false, true, false, false},
            {false, true, true, false},
    };

    public static void main(String[] args) {
        int players = Integer.parseInt(option(args, "--player", "1"));
        int npcs = Integer.parseInt(option(args, "--npc", "1"));
        long seed = Long.parseLong(option(args, "--seed",
                String.valueOf(System.nanoTime())));
        if (players < 0 || npcs < 0 || players + npcs < 2) {
            printUsage();
            return;
        }
        Random random = new Random(seed);
        Scanner scanner = new Scanner(System.in);

        List<String> members = createMembers(players, npcs);
        int round = 1;
```

JankenApp.java（続き）

```
        while (members.size() > 1) {
            printRoundHeader(round, members);
            List<Integer> hands =
                    chooseHands(members, scanner, random);
            printHands(members, hands);
            members = survivors(members, hands);
            if (members.size() > 1) {
                System.out.println("Draw: remaining " + members);
            }
            round++;
        }
        System.out.println();
        System.out.println("The winner is " + members.get(0) + "!");
    }

    static String option(String[] args, String name,
            String defaultValue) {
        for (int i = 0; i + 1 < args.length; i += 2) {
            if (args[i].equals(name)) {
                return args[i + 1];
            }
        }
        return defaultValue;
    }
```

解答例 ・ 課題3（2/3）

メソッドに切り出した例

JankenApp.java（続き）

```
    static void printUsage() {
        System.out.println("Usage: JankenApp [--player N]"
                + " [--npc N] [--seed N]");
        System.out.println("  N >= 0, 2 or more players in total");
    }

    static List<String> createMembers(int players, int npcs) {
        List<String> members = new ArrayList<>();
        for (int i = 1; i <= players; i++) {
            members.add("Player" + i);
        }
        for (int i = 1; i <= npcs; i++) {
            members.add("Npc" + i);
        }
        return members;
    }

    static void printRoundHeader(int round, List<String> members) {
        System.out.println();
        System.out.println("--- Round " + round + " ("
                + members.size() + " players left) ---");
    }

    static List<Integer> chooseHands(List<String> members,
            Scanner scanner, Random random) {
```

JankenApp.java（続き）

```
        List<Integer> hands = new ArrayList<>();
        for (String name : members) {
            if (name.startsWith("Player")) {
                hands.add(askHand(name, scanner));
            } else {
                hands.add(random.nextInt(NAMES.length));
            }
        }
        return hands;
    }

    static int askHand(String name, Scanner scanner) {
        while (true) {
            System.out.print(name + ", choose your hand"
                    + " [r]ock, [p]aper, [s]cissors, [j]ackal > ");
            String input = scanner.nextLine().trim();
            for (int h = 0; h < KEYS.length; h++) {
                if (KEYS[h].equals(input)) {
                    return h;
                }
            }
        }
    }

    static void printHands(List<String> members,
```

解答例 ・ 課題3（3/3）

メソッドに切り出した例

JankenApp.java（続き）

```
            List<Integer> hands) {
        for (int i = 0; i < members.size(); i++) {
            System.out.println("  " + members.get(i) + ": "
                    + NAMES[hands.get(i)]);
        }
    }

    static boolean isBeaten(int hand, List<Integer> hands) {
        for (int other : hands) {
            if (BEATS[other][hand]) {
                return true;
            }
        }
        return false;
    }

    static List<String> survivors(List<String> members,
            List<Integer> hands) {
        List<String> survivors = new ArrayList<>();
        for (int i = 0; i < members.size(); i++) {
            if (!isBeaten(hands.get(i), hands)) {
                survivors.add(members.get(i));
            }
        }
        if (survivors.isEmpty()) {
```

JankenApp.java（続き）

```
                return members;
            }
            return survivors;
        }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 課題4

クラスに切り出す

課題3で、いつも同じデータを受け取っていたメソッドを、そのデータと一緒にクラスへまとめます。int の手も、手を表すものに置き換えます。

1. フィールドはすべて private にし、外から直接変更できないようにする
2. 手どうしの勝ち負けを if や switch で書き分けず、手ごとに「自分を負かす手」を持たせる
3. 起動時の人数は 0 以上で、合計 2 名以上（満たさない設定は作れない）
4. だれがどの手を出したかは、必ず組になっている

ヒント：手は課題3と同じ順（Rock, Paper, Scissors, Jackal）に並べると、同じシードで同じ表示になる

検討例 ・ 課題4

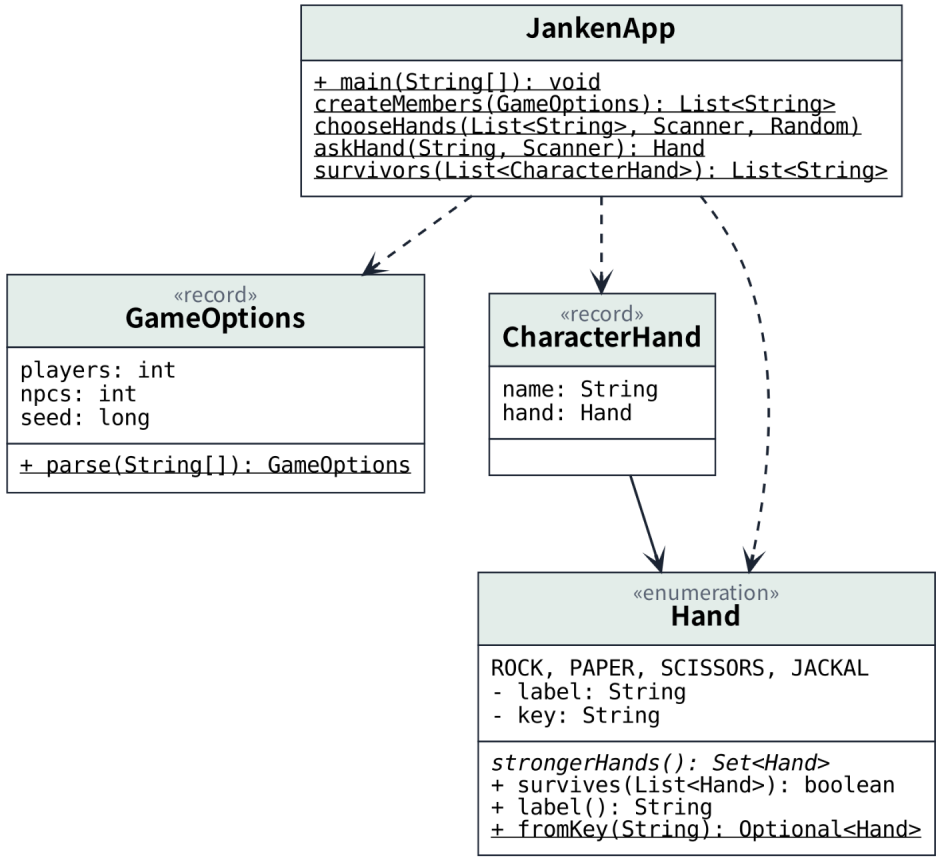
クラスに切り出す

役割	作るもの
4種類の手と、表示名・入力の文字	enum Hand：ROCK, PAPER, SCISSORS, JACKAL
自分を負かす手を教える（手ごとに違う）	Hand：strongerHands(): Set<Hand>
場の手と比べて、自分が残れるかを検査	Hand：survives(List<Hand>): boolean
入力された文字に対応する手を探す	Hand：static fromKey(String): Optional<Hand>
人数とシードを組で持つ（条件を守る）	record GameOptions(players, npcs, seed)
起動引数から設定を作る	GameOptions：static parse(String[])
だれがどの手を出したかを組で持つ	record CharacterHand(String name, Hand hand)

JankenApp には、参加者の並び・手の出し方・残る人の決め方が残る

検討例 ・ 課題4 クラス図

データと処理を、まとめて閉じ込める



«enumeration» Hand

どの手に負けるかを、定数ごとに知っている

«record» GameOptions

人数とシードを組で持ち、条件を満たさない設定は作れない

«record» CharacterHand

だれが、どの手を出したかを組で持つ

JankenApp

参加者の並び・手の出し方・残る人の決め方が、まだ残っている

解答例 ・ 課題4（1/4）

クラスに切り出した例

Hand.java

```
/** じゃんけんの手。どの手に負けるかを、手自身が知っている。 */
public enum Hand {
    ROCK("Rock", "r") {
        @Override
        Set<Hand> strongerHands() {
            return EnumSet.of(PAPER);
        }
    },
    PAPER("Paper", "p") {
        @Override
        Set<Hand> strongerHands() {
            return EnumSet.of(SCISSORS, JACKAL);
        }
    },
    SCISSORS("Scissors", "s") {
        @Override
        Set<Hand> strongerHands() {
            return EnumSet.of(ROCK, JACKAL);
        }
    },
    JACKAL("Jackal", "j") {
        @Override
        Set<Hand> strongerHands() {
            return EnumSet.of(ROCK);
        }
```

Hand.java（続き）

```
    };

    private final String label;
    private final String key;

    Hand(String label, String key) {
        this.label = label;
        this.key = key;
    }

    /** 自分を負かす手 */
    abstract Set<Hand> strongerHands();

    /** 場に自分より強い手が出ていなければ、残れる */
    public boolean survives(List<Hand> field) {
        for (Hand other : field) {
            if (strongerHands().contains(other)) {
                return false;
            }
        }
        return true;
    }

    public String label() {
        return label;
```

解答例 ・ 課題4（2/4）

クラスに切り出した例

Hand.java（続き）

```
    }

    public static Optional<Hand> fromKey(String key) {
        for (Hand hand : values()) {
            if (hand.key.equals(key)) {
                return Optional.of(hand);
            }
        }
        return Optional.empty();
    }
}
```

GameOptions.java

```
/** 起動時の設定。人数は 0 以上で、合計 2 名以上。 */
public record GameOptions(int players, int npcs, long seed) {
    public GameOptions {
        if (players < 0 || npcs < 0 || players + npcs < 2) {
            throw new IllegalArgumentException(
                    "N >= 0, 2 or more players in total");
        }
    }

    public static GameOptions parse(String[] args) {
        int players = Integer.parseInt(option(args, "--player", "1"));
        int npcs = Integer.parseInt(option(args, "--npc", "1"));
        long seed = Long.parseLong(option(args, "--seed",
                String.valueOf(System.nanoTime())));
        return new GameOptions(players, npcs, seed);
    }

    private static String option(String[] args, String name,
            String defaultValue) {
        for (int i = 0; i + 1 < args.length; i += 2) {
            if (args[i].equals(name)) {
                return args[i + 1];
            }
        }
        return defaultValue;
    }
}
```

CharacterHand.java

```
/** だれが、どの手を出したか。どちらも欠けてはいけない。 */
public record CharacterHand(String name, Hand hand) {
    public CharacterHand {
        Objects.requireNonNull(name);
        Objects.requireNonNull(hand);
    }
}
```

解答例 ・ 課題4（3/4）

クラスに切り出した例

JankenApp.java

```
public class JankenApp {
    public static void main(String[] args) {
        GameOptions options;
        try {
            options = GameOptions.parse(args);
        } catch (IllegalArgumentException e) {
            System.out.println("Usage: JankenApp [--player N]"
                    + " [--npc N] [--seed N]");
            System.out.println("  " + e.getMessage());
            return;
        }
        Random random = new Random(options.seed());
        Scanner scanner = new Scanner(System.in);

        List<String> members = createMembers(options);
        int round = 1;
        while (members.size() > 1) {
            System.out.println();
            System.out.println("--- Round " + round + " ("
                    + members.size() + " players left) ---");
            List<CharacterHand> played =
                    chooseHands(members, scanner, random);
            for (CharacterHand ch : played) {
                System.out.println("  " + ch.name() + ": "
                        + ch.hand().label());
```

JankenApp.java（続き）

```
            }
            members = survivors(played);
            if (members.size() > 1) {
                System.out.println("Draw: remaining " + members);
            }
            round++;
        }
        System.out.println();
        System.out.println("The winner is " + members.get(0) + "!");
    }

    static List<String> createMembers(GameOptions options) {
        List<String> members = new ArrayList<>();
        for (int i = 1; i <= options.players(); i++) {
            members.add("Player" + i);
        }
        for (int i = 1; i <= options.npcs(); i++) {
            members.add("Npc" + i);
        }
        return members;
    }

    static List<CharacterHand> chooseHands(List<String> members,
            Scanner scanner, Random random) {
        List<CharacterHand> played = new ArrayList<>();
```

解答例 ・ 課題4（4/4）

クラスに切り出した例

JankenApp.java（続き）

```
        for (String name : members) {
            Hand hand;
            if (name.startsWith("Player")) {
                hand = askHand(name, scanner);
            } else {
                Hand[] all = Hand.values();
                hand = all[random.nextInt(all.length)];
            }
            played.add(new CharacterHand(name, hand));
        }
        return played;
    }

    static Hand askHand(String name, Scanner scanner) {
        while (true) {
            System.out.print(name + ", choose your hand"
                    + " [r]ock, [p]aper, [s]cissors, [j]ackal > ");
            Optional<Hand> hand =
                    Hand.fromKey(scanner.nextLine().trim());
            if (hand.isPresent()) {
                return hand.get();
            }
        }
    }
```

JankenApp.java（続き）

```
    static List<String> survivors(List<CharacterHand> played) {
        List<Hand> field = new ArrayList<>();
        for (CharacterHand ch : played) {
            field.add(ch.hand());
        }
        List<String> survivors = new ArrayList<>();
        List<String> all = new ArrayList<>();
        for (CharacterHand ch : played) {
            all.add(ch.name());
            if (ch.hand().survives(field)) {
                survivors.add(ch.name());
            }
        }
        return survivors.isEmpty() ? all : survivors;
    }
}
```

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 課題5

役割で分ける

chooseHands() には、名前が Player で始まるかどうかで、手の出し方を書き分ける if が残っています。違いをそれぞれに任せ、じゃんけんの進行もクラスにします。

1. プレイヤーとNPCに共通する参加者の型を作り、手の出し方をそれぞれに任せる
2. だれがどの手を出したかを、名前ではなく参加者そのものと組にする
3. 勝者が決まるまで繰り返すクラスを作り、参加者が2名以上でなければ作れないようにする
4. main は、起動の設定を読み、参加者をそろえて始めるだけにする

条件：じゃんけんを進めるクラスは、相手が Player か Npc かを知らずに手を出させる

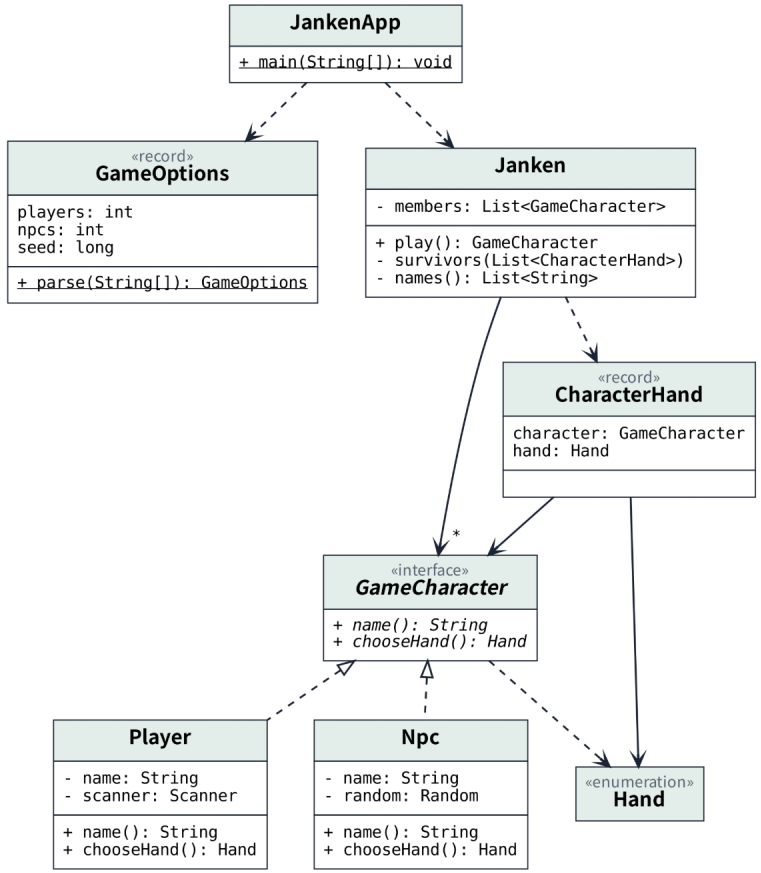
検討例 ・ 課題5

役割で分ける

役割	作るもの
プレイヤーとNPCに共通する参加者の型	interface GameCharacter：name(), chooseHand()
画面から手を入力してもらう	Player implements GameCharacter
4種類の手からランダムに選ぶ	Npc implements GameCharacter
だれがどの手を出したかを組で持つ	record CharacterHand(GameCharacter, Hand)
参加者を受け取る（2名以上でなければ例外）	Janken：コンストラクタ
勝者が決まるまで繰り返し、勝者を返す	Janken：play(): GameCharacter
残る人を決める（全員負けたら全員）	Janken：survivors()（private）

検討例 ・ 課題5 クラス図

違いは、それぞれの参加者に任せる



«interface» GameCharacter

名前と手の出し方を決める。実装はそれぞれの参加者

白抜きの三角と破線

Player と Npc は GameCharacter を実装する

Janken

参加者の種類を区別せずに chooseHand() を呼ぶ

Hand

中身は課題4と同じ（図では省略）

解答例 ・ 課題5（1/4）

役割で分けた例

GameCharacter.java

```
/** じゃんけんの参加者。手の決め方は参加者ごとに違う。 */
public interface GameCharacter {
    String name();

    Hand chooseHand();
}
```

Player.java

```
/** 人。画面から手を入力してもらう。 */
public class Player implements GameCharacter {
    private final String name;
    private final Scanner scanner;

    public Player(String name, Scanner scanner) {
        this.name = name;
        this.scanner = scanner;
    }

    @Override
    public String name() {
        return name;
    }

    @Override
    public Hand chooseHand() {
```

Player.java（続き）

```
        while (true) {
            System.out.print(name + ", choose your hand"
                    + " [r]ock, [p]aper, [s]cissors, [j]ackal > ");
            Optional<Hand> hand =
                    Hand.fromKey(scanner.nextLine().trim());
            if (hand.isPresent()) {
                return hand.get();
            }
        }
    }
}
```

Npc.java

```
/** コンピューター。4 種類の手からランダムに選ぶ。 */
public class Npc implements GameCharacter {
    private final String name;
    private final Random random;

    public Npc(String name, Random random) {
        this.name = name;
        this.random = random;
    }

    @Override
    public String name() {
```

解答例 ・ 課題5（2/4）

役割で分けた例

Npc.java（続き）

```
        return name;
    }

    @Override
    public Hand chooseHand() {
        Hand[] all = Hand.values();
        return all[random.nextInt(all.length)];
    }
}
```

CharacterHand.java

```
/** だれが、どの手を出したか。どちらも欠けてはいけない。 */
public record CharacterHand(GameCharacter character, Hand hand) {
    public CharacterHand {
        Objects.requireNonNull(character);
        Objects.requireNonNull(hand);
    }
}
```

Janken.java

```
/** 勝者が一人に決まるまで、じゃんけんを繰り返す */
public class Janken {
    private List<GameCharacter> members;

    public Janken(List<GameCharacter> members) {
        if (members.size() < 2) {
            throw new IllegalArgumentException("2 or more players");
        }
        this.members = List.copyOf(members);
    }

    public GameCharacter play() {
        int round = 1;
        while (members.size() > 1) {
            System.out.println();
            System.out.println("--- Round " + round + " ("
                    + members.size() + " players left) ---");
            List<CharacterHand> played = new ArrayList<>();
            for (GameCharacter member : members) {
                played.add(new CharacterHand(member,
                        member.chooseHand()));
            }
            for (CharacterHand ch : played) {
                System.out.println("  " + ch.character().name()
                        + ": " + ch.hand().label());
```

解答例 ・ 課題5（3/4）

役割で分けた例

Janken.java（続き）

```
            }
            members = survivors(played);
            if (members.size() > 1) {
                System.out.println("Draw: remaining " + names());
            }
            round++;
        }
        return members.get(0);
    }

    /** 負けなかった人が残る。全員が負けたら、全員が残る */
    private List<GameCharacter> survivors(
            List<CharacterHand> played) {
        List<Hand> field = new ArrayList<>();
        for (CharacterHand ch : played) {
            field.add(ch.hand());
        }
        List<GameCharacter> survivors = new ArrayList<>();
        for (CharacterHand ch : played) {
            if (ch.hand().survives(field)) {
                survivors.add(ch.character());
            }
        }
        return survivors.isEmpty() ? members : survivors;
    }
```

Janken.java（続き）

```
    private List<String> names() {
        List<String> names = new ArrayList<>();
        for (GameCharacter member : members) {
            names.add(member.name());
        }
        return names;
    }
}
```

JankenApp.java

```
public class JankenApp {
    public static void main(String[] args) {
        GameOptions options;
        try {
            options = GameOptions.parse(args);
        } catch (IllegalArgumentException e) {
            System.out.println("Usage: JankenApp [--player N]"
                    + " [--npc N] [--seed N]");
            System.out.println("  " + e.getMessage());
            return;
        }
        Random random = new Random(options.seed());
        Scanner scanner = new Scanner(System.in);

        List<GameCharacter> members = new ArrayList<>();
```

解答例 ・ 課題5（4/4）

役割で分けた例

JankenApp.java（続き）

```
        for (int i = 1; i <= options.players(); i++) {
            members.add(new Player("Player" + i, scanner));
        }
        for (int i = 1; i <= options.npcs(); i++) {
            members.add(new Npc("Npc" + i, random));
        }

        GameCharacter winner = new Janken(members).play();
        System.out.println();
        System.out.println("The winner is " + winner.name() + "!");
    }
}
```

課題

実装で意識すること

プラクティス	このゲームでの例
責務の分割 1つのクラスは1つのことを知る	勝ち負けは Hand、手の出し方は Player と Npc 残る人の決め方と回の繰り返しは Janken、起動の設定は GameOptions
カプセル化 不変条件を守る	参加者は2名以上、起動時の人数は0以上で合計2名以上 だれがどの手を出したかは、必ず組になっている
多態性 種類を知らずに任せる	Janken は Player と Npc を区別せずに chooseHand() を呼ぶ 勝ち負けは、それぞれの手が知っている

データとそれを扱う処理は一緒に置き、種類ごとの違いはそれぞれに任せる。

課題 ・ ふりかえり

最初の main と比べてみよう

課題2と課題5のコードを並べて、次のことを話し合いましょう。

1. 新しい手を1つ加えるとき、それぞれどこを直すか

2. 毎回同じ手を出す NPC を加えるとき、それぞれどこを直すか

3. 課題4・5で分けた境目は、課題1で見つけた「もの」と同じだったか

分けた境目が、変更とテストのしやすさにつながっている。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

JankenApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・振り返る

早く終わった人向け

演習 ・ 発展課題

仕様を変えてみる

早く終わった人は、次の変更に挑戦しましょう。変更したら、直したクラスとメソッドの数を数えてください。

1. 新しい手を1つ追加する（どの手より強いかは、自分で決める）
2. 毎回同じ手を出す NPC を追加する
3. 2で追加した NPC の人数も、起動引数で指定できるようにする

直す箇所が少なく、1か所に集まっているほど、変更に強い設計です。Janken を直す必要があったかどうかにも注目しましょう。

付録 ・ あとがき・考えてみよう（1/2）

見つけた法則と、実装の対応を振り返る

課題1で見つけた法則	対応するメソッド
3. 残れる ＝ 自分より強い手が場にない	Hand.survives()
3. 全員が負けたら、全員が残る（3すくみ）	Janken.survivors()

解答例では、「自分より強い手が場にないか」は手（Hand）に、「全員が負けたら全員が残る」はじゃんけん（Janken）に任せました。

一方で、3すくみも手どうしの勝ち負けから決まるので、手の側にまとめて「場に出た手の中から、残る手を選ぶ」役割を Hand に持たせる、という選択もあり得ます。

3すくみの判定は、Hand と Janken のどちらに任せるべきでしょうか。

付録 ・ あとがき・考えてみよう（2/2）

どちらを選んでも、説明できれば正しい

Hand に任せる

3すくみは「手のルール」

勝ち負けに関わることは、すべて手が知っている。Janken は、ゲームの継続と決着だけを管理すればよくなる。

Janken に任せる（解答例）

3すくみは「じゃんけん全体のルール」

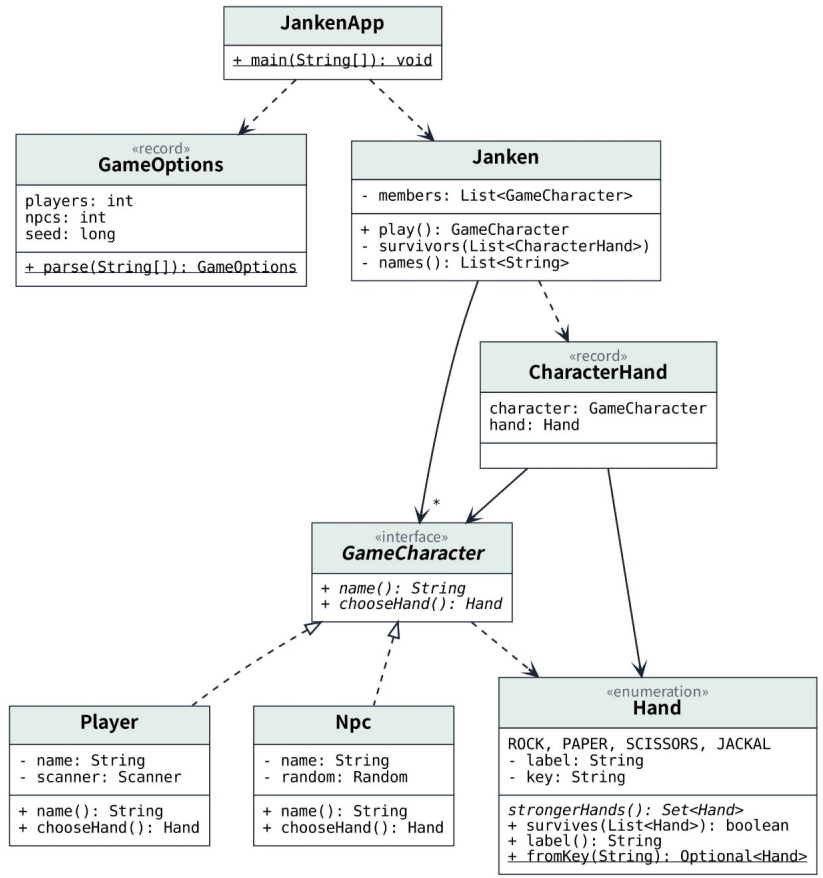
手そのものではなく、手を使って行うじゃんけん全体に適用される決まりとして扱う。

どちらの選択も、間違いではありません。概念をどう捉えたかによって、モデルの形は変わります。概念を観察して考え、選んだ理由を説明できるなら、それは正しい実装です。反対に、「動いたからこうした」としか言えないなら、動いていても誤りです。

このことを意識して、これからもモデリングに取り組んでください。

検討例 ・ 付録 クラス図

全体のつながり



JankenApp・GameOptions

起動引数を読み、参加者をそろえて始める

Janken

勝者が一人に決まるまで、回を繰り返す

GameCharacter・Player・Npc

手の出し方だけを、それぞれに任せる（課題5）

Hand・CharacterHand

勝ち負けと、だれがどの手を出したか（課題4）